

PROYECTO

Martin Guliano Medina Urraca 3490 25 16868

Ingeniería en sistemas Mariano Gálvez sede San José Pinula

Programación II

17 de octubre de 2026

ÍNDICE

Portada **1**

Introducción **1.1**

Virtualización y entorno de trabajo **2**

Justificación de los recursos asignados **3**

Instalación y configuración del servidor **4**

Configuración del VirtualHost **5**

Diagrama de casos de uso **6**

Arquitectura del sistema **7**

Arquitectura del proyecto original en PHP 7.1

Descripción de los componentes del sistema original 7.2

Arquitectura propuesta para CarritoGT con Spring Boot 7.3

Diseño de la base de datos **8**

Descripción general de la base de datos 8.1

Diccionario de datos 8.3

Tipos de datos y restricciones 8.4

Contrato de la API REST **9**

Pruebas y evidencias de funcionamiento **10**

Objetivo de las pruebas **10.1**

Pruebas automatizadas con Maven 10.2

Prueba de consulta de productos 10.3

Manual de usuario **11**

Bitácora de desarrollo **12**

Registro cronológico de actividades 12.1

Avances realizados por fecha 12.2

Dificultades encontradas y soluciones aplicadas 12.3

Introducción

El presente documento describe el desarrollo del proyecto integrador CarritoGT, cuyo objetivo es analizar un sistema de carrito de compras desarrollado en PHP y trabajar en una propuesta basada en una API REST con Spring Boot. Durante el proceso se documentan la configuración del entorno de trabajo, la arquitectura del sistema, el diseño de la base de datos, los servicios de la API y las pruebas realizadas. Asimismo, se presentan las evidencias y la bitácora de trabajo para registrar los avances y resultados obtenidos durante el desarrollo del proyecto.

Proyecto CarritoGT

2.Evidencias de Virtualización

La virtualización permite ejecutar un sistema operativo dentro de otro sin necesidad de instalarlo directamente en el equipo físico. Para este proyecto, el objetivo de utilizar una máquina virtual es contar con un entorno GNU/Linux donde se puedan instalar y configurar las herramientas necesarias para trabajar con servidores web, bases de datos y control de versiones.

El equipo físico utiliza Windows, mientras que la máquina virtual se plantea con Lubuntu como sistema operativo invitado. Este entorno permite realizar prácticas de administración de servidores y desarrollo de software sin modificar directamente la instalación principal de Windows.

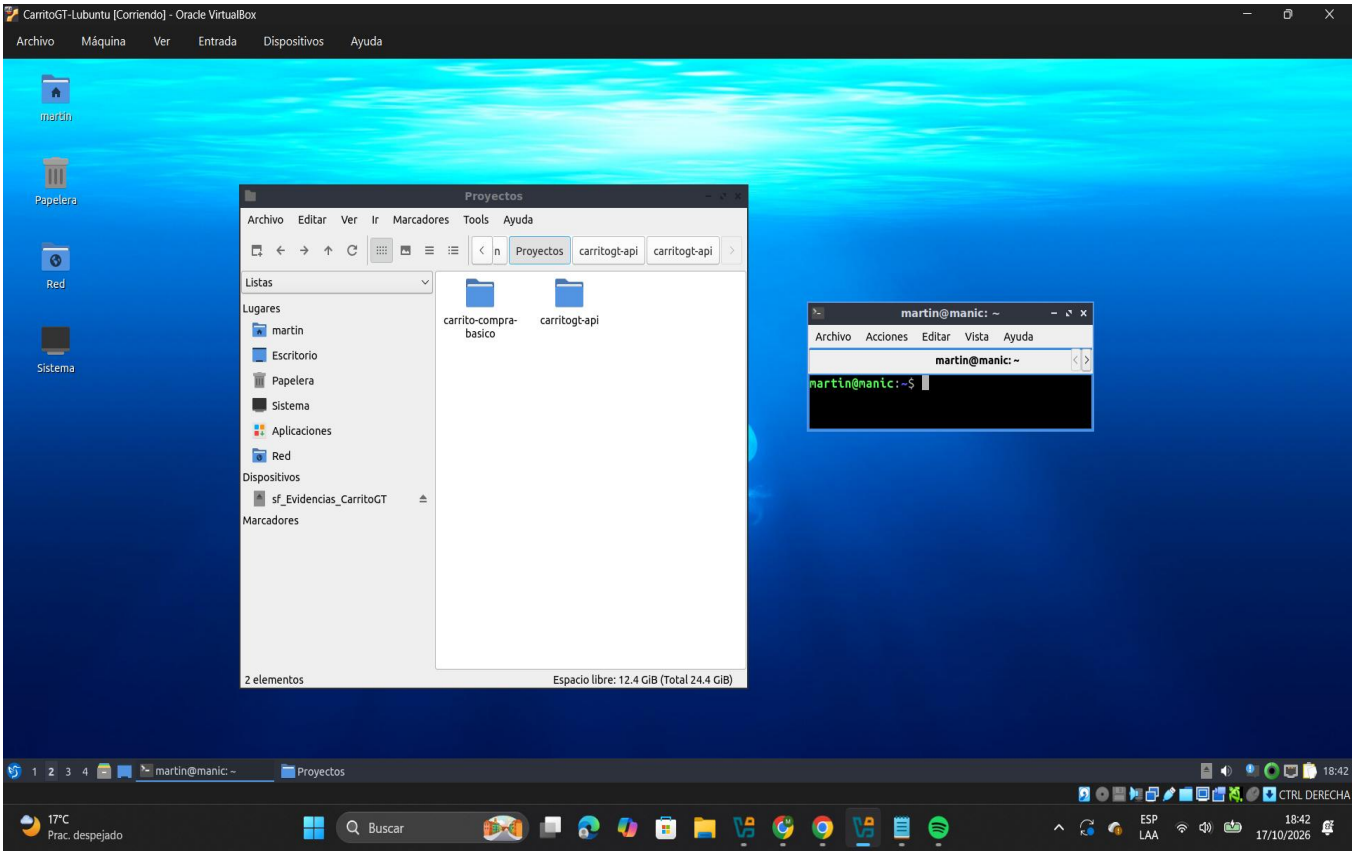


Figura 1. Programa de virtualización utilizado VirtualBox

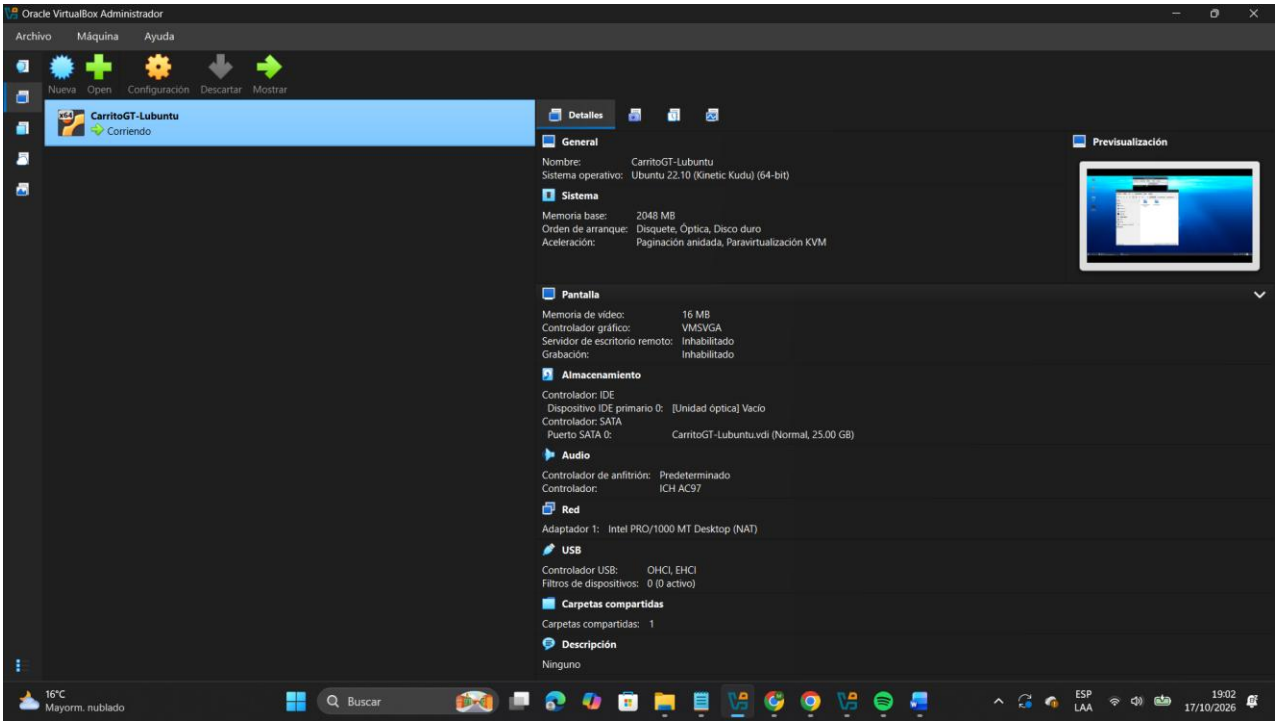


Figura 2. Configuración de la máquina virtual, incluyendo memoria RAM, procesadores y disco virtual.

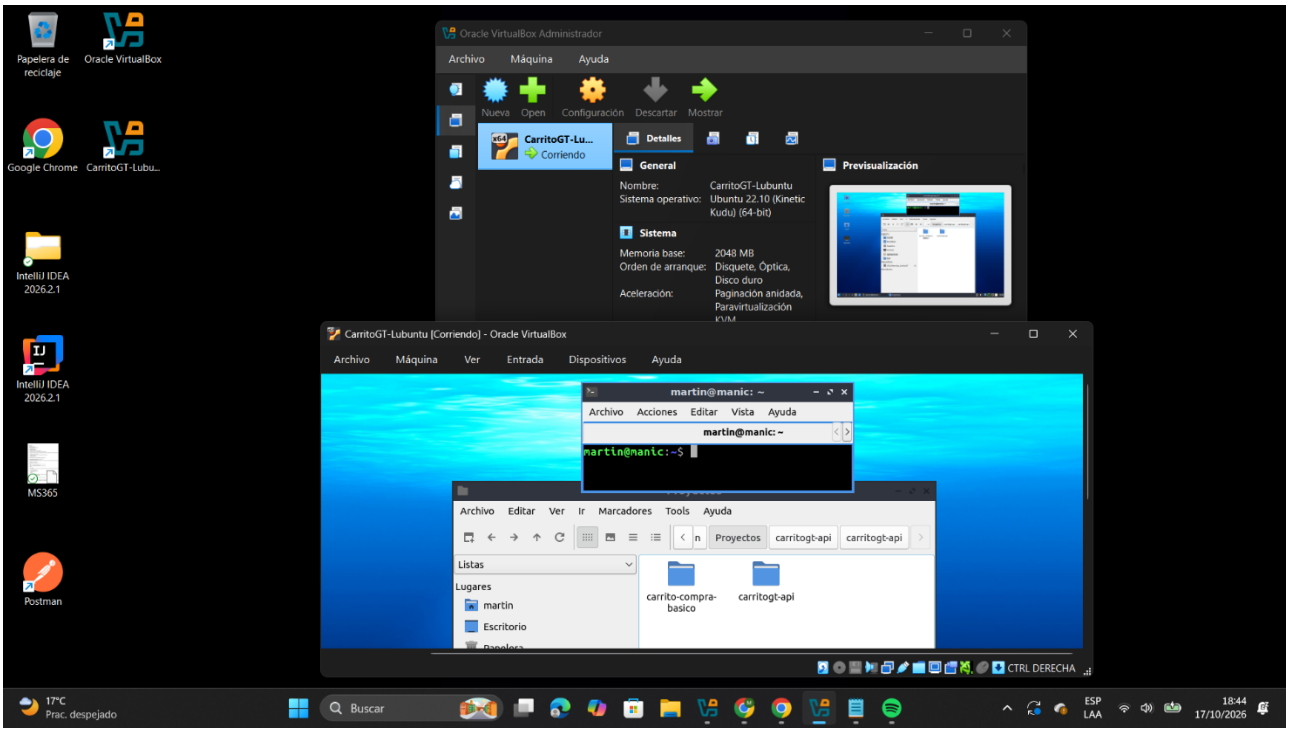


Figura 3. Windows y Lubuntu funcionando al mismo tiempo, si se logró realizar la práctica.

3. Justificación de los recursos de la máquina virtual

3.1. Memoria RAM

La memoria RAM se utiliza para mantener en funcionamiento el sistema operativo y los programas abiertos. Se debe asignar una cantidad suficiente para que Lubuntu pueda ejecutar las herramientas del proyecto, como Git, Apache, PHP, MySQL o MariaDB y las herramientas de desarrollo.

Se recomienda comenzar con 2 GB de RAM para una máquina virtual ligera, si el equipo físico cuenta con memoria suficiente. Si se necesita utilizar varias herramientas al mismo tiempo y los recursos del equipo lo permiten, se puede considerar una asignación mayor.

3.2. Procesador

Los procesadores asignados a la máquina virtual permiten ejecutar las tareas de instalación, compilación y pruebas. Se recomienda iniciar con 2 procesadores virtuales, siempre que el equipo físico tenga capacidad suficiente. La asignación debe evitar que Windows y los demás programas se vuelvan demasiado lentos.

3.3. Almacenamiento

El disco virtual debe contar con espacio para el sistema operativo, las herramientas de desarrollo, el código fuente, la base de datos y los archivos de evidencia. Una capacidad inicial de 25 a 30 GB puede servir para un entorno básico, aunque debe ajustarse de acuerdo con las necesidades de la instalación.

3.4. Red

La configuración de red permite que la máquina virtual se conecte a Internet para descargar paquetes, clonar repositorios y actualizar herramientas. También debe permitir realizar pruebas de acceso al servidor local y a la aplicación.

El modo NAT puede utilizarse para proporcionar acceso a Internet. Si se necesita acceder a servicios de la máquina virtual desde Windows u otros equipos, se debe evaluar una configuración adicional, como el reenvío de puertos o una red puente.

3.5. Estabilidad y ajustes

Los recursos deben asignarse de forma equilibrada para evitar que la máquina virtual consuma toda la capacidad del equipo físico. También es conveniente mantener actualizado el sistema operativo, cerrar programas innecesarios y guardar capturas de la configuración utilizada.

Configuración de Referencia Propuesta

Recurso	Asignación Inicial Propuesta
Sistema Operativo Invitado	**Lubuntu GNU/Linux**
Memoria RAM	**2GB**
Procesadores Virtuales	**2**
Disco Virtual	**25-30GB**
Red	**NAT para acceso a internet**

4. Evidencias del servidor y herramientas instaladas

4.1. Descripción

Para desarrollar y probar el proyecto se necesitan herramientas que permitan administrar el código fuente, ejecutar aplicaciones web y gestionar bases de datos. Entre las herramientas contempladas para el entorno GNU/Linux se encuentran Git, Apache, PHP y MySQL o MariaDB.

Git permite registrar los cambios del proyecto y compartir el código mediante GitHub. Apache funciona como servidor web para la versión PHP del proyecto. PHP ejecuta la lógica del sistema original y MySQL o MariaDB permite almacenar la información de los productos y del carrito.

En la versión desarrollada con Java, Spring Boot permite crear los servicios web que reciben solicitudes HTTP y responden con información en formato JSON.

4.2. Comandos de verificación

En Lubuntu, si las herramientas ya están instaladas, se pueden ejecutar los siguientes comandos en la terminal:

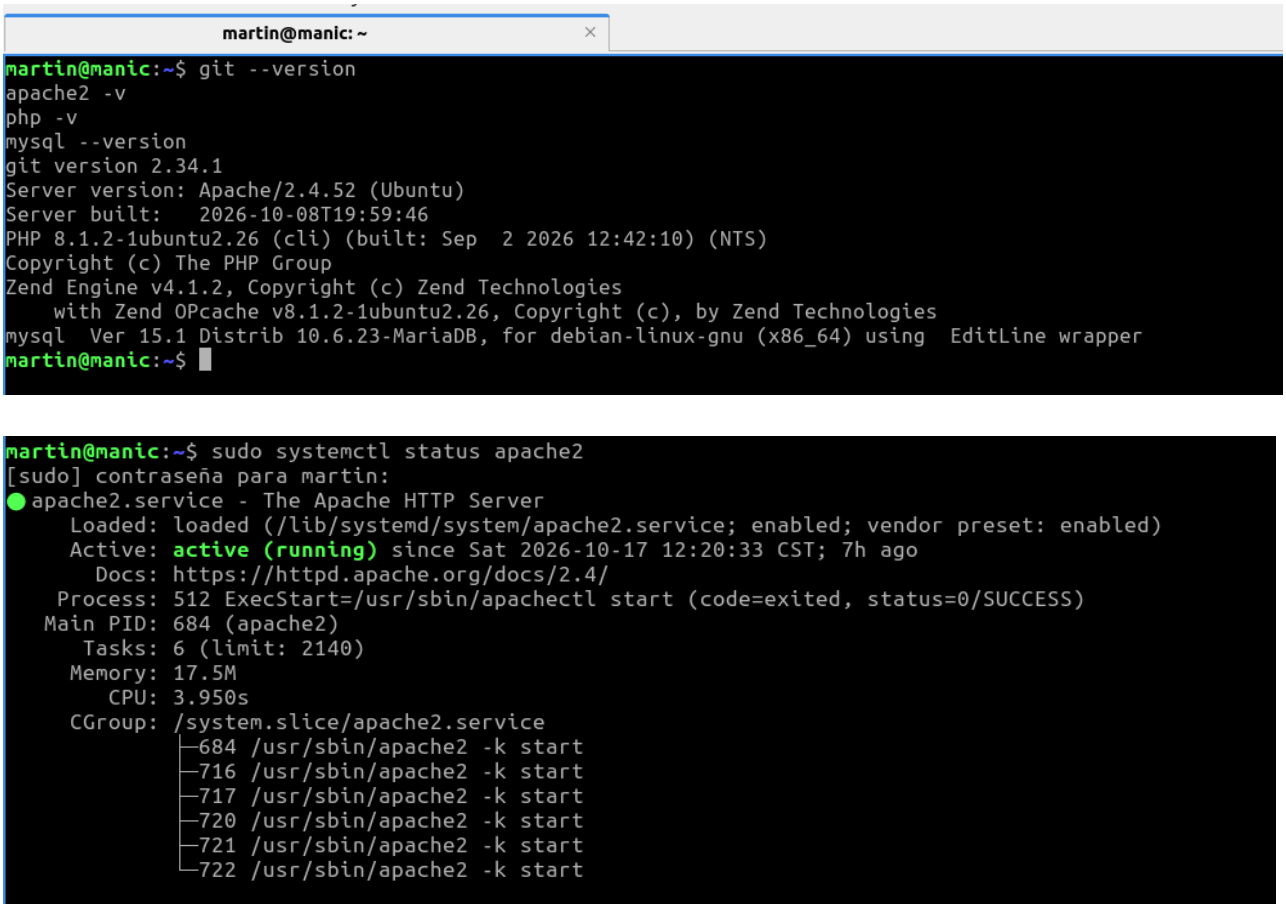


Figura 4. Para comprobar el estado de Apache.

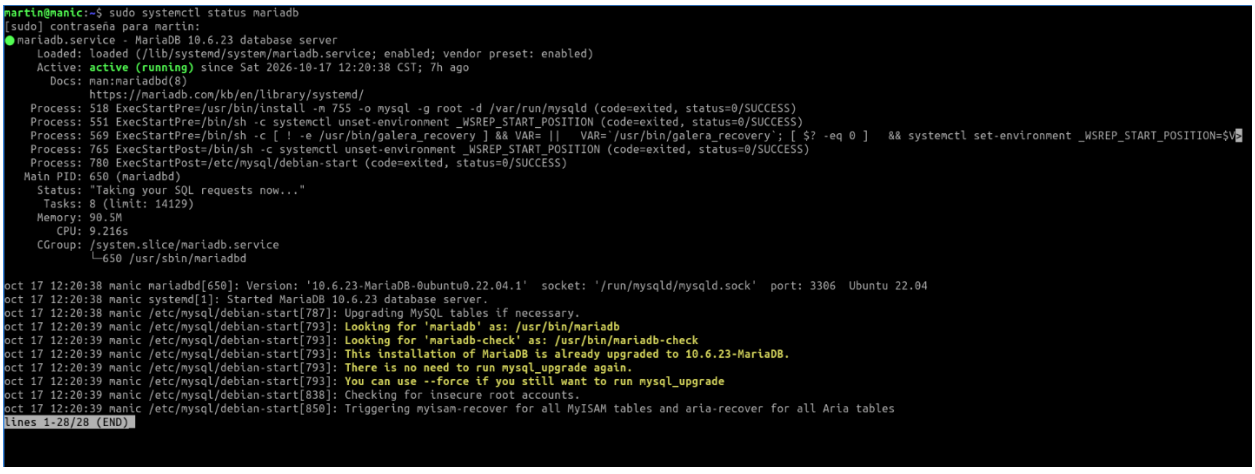


Figura 5. Para comprobar el estado de MariaDB, si ese es el servicio instalad

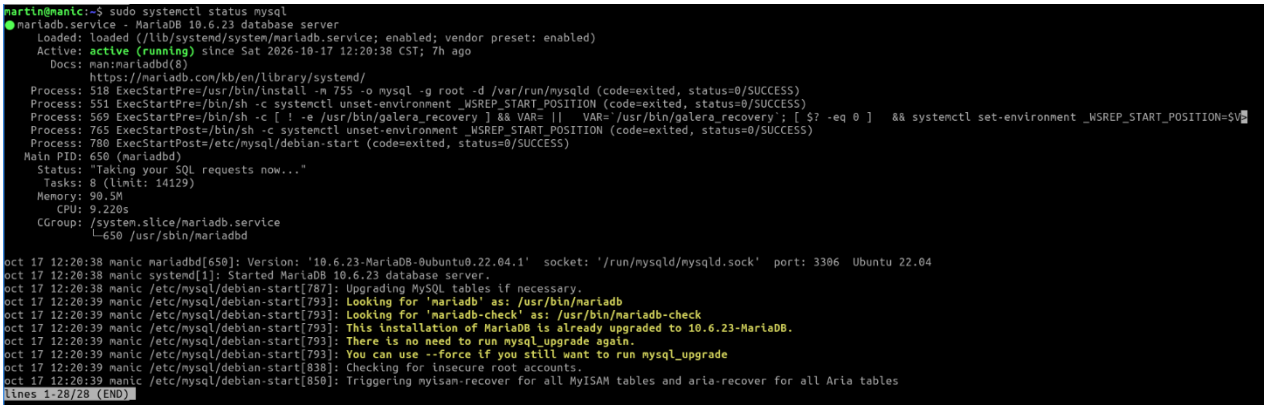


Figura 6. Si se instaló MySQL en lugar de MariaDB, se debe comprobar el servicio correspondiente.

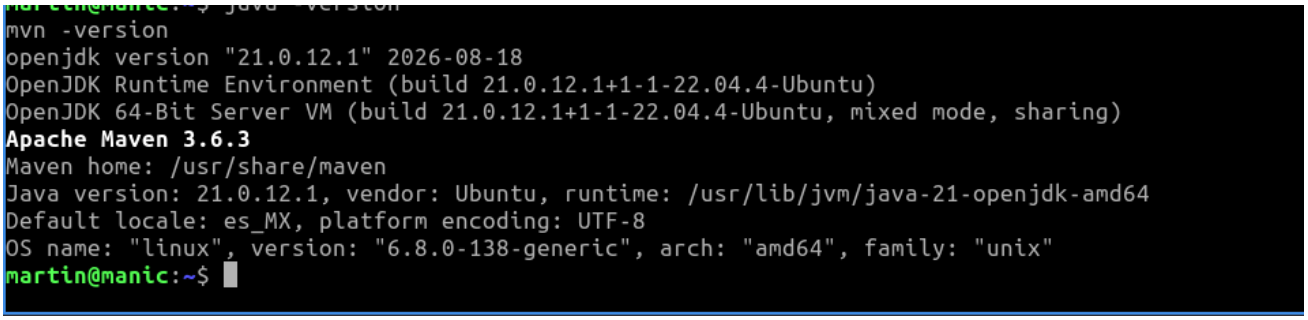


Figura 7. Para comprobar la versión de Java y Maven utilizados en el proyecto Spring Boot.

5. Configuración del Virtual Host

Se configuró un Virtual Host en Apache con el propósito de facilitar el acceso a la aplicación PHP mediante una dirección local. Para ello, se utilizó un archivo de configuración en el que se definieron el nombre del sitio, la carpeta del proyecto y los permisos de acceso.

Posteriormente, se verificó la sintaxis de la configuración y se recargó el servicio Apache para aplicar los cambios. Finalmente, se realizó una prueba desde el navegador para comprobar el acceso a la aplicación.

La dirección de referencia utilizada fue http://carrito-compra.localhost.

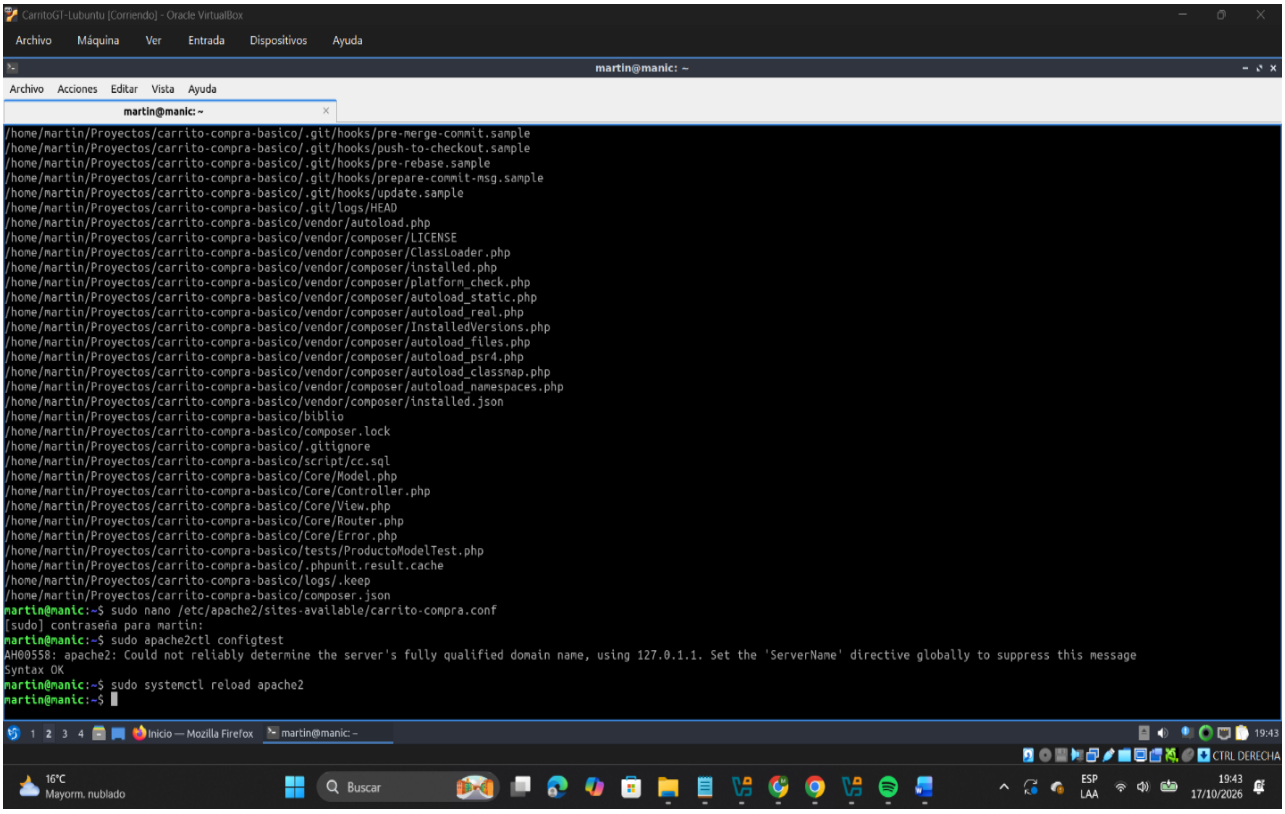


Figura 8. Resultado de apache2ctl configtest.

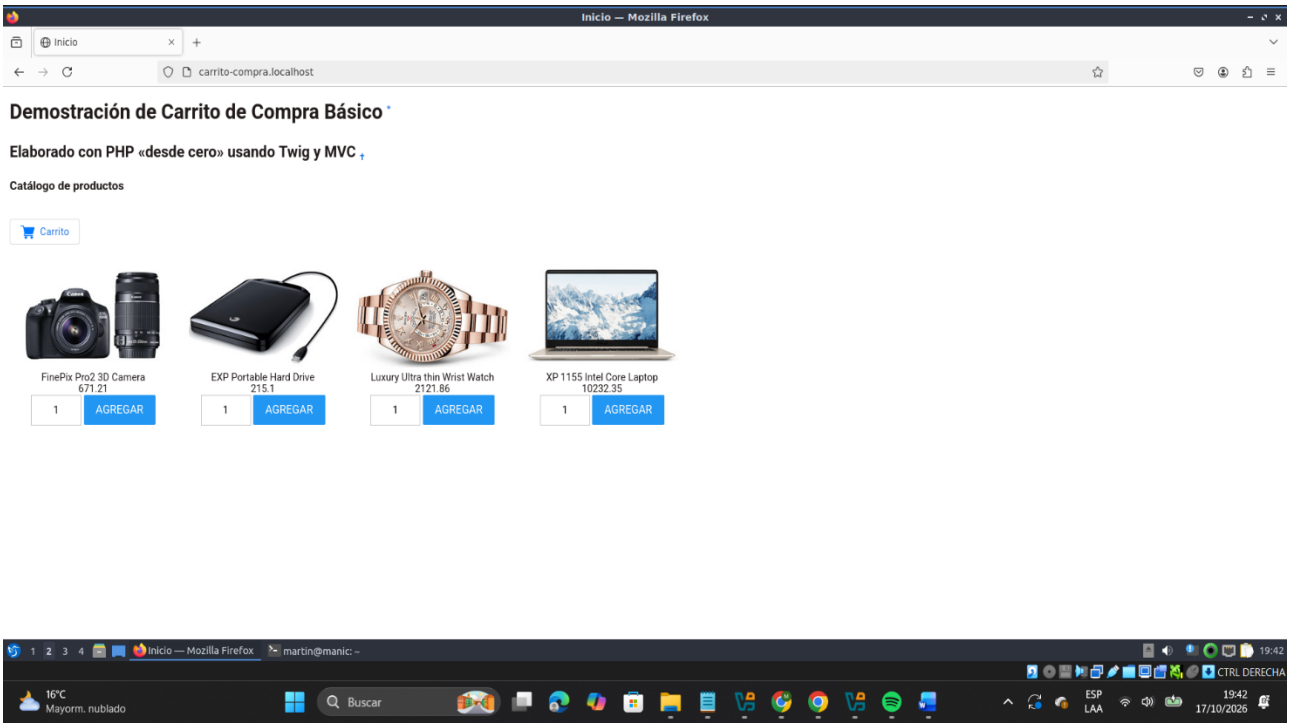


Figura 9. Aplicación PHP abierta en el navegador

6. Diagrama de casos de uso

6.1. Descripción

El diagrama de casos de uso representa las acciones principales que puede realizar una persona al utilizar el carrito de compras. El actor principal es el usuario, quien interactúa con el sistema para consultar productos y administrar los artículos seleccionados.

Los casos de uso considerados son:

- Consultar o visualizar productos.
- Agregar productos al carrito.
- Consultar los productos agregados al carrito.
- Eliminar productos del carrito.

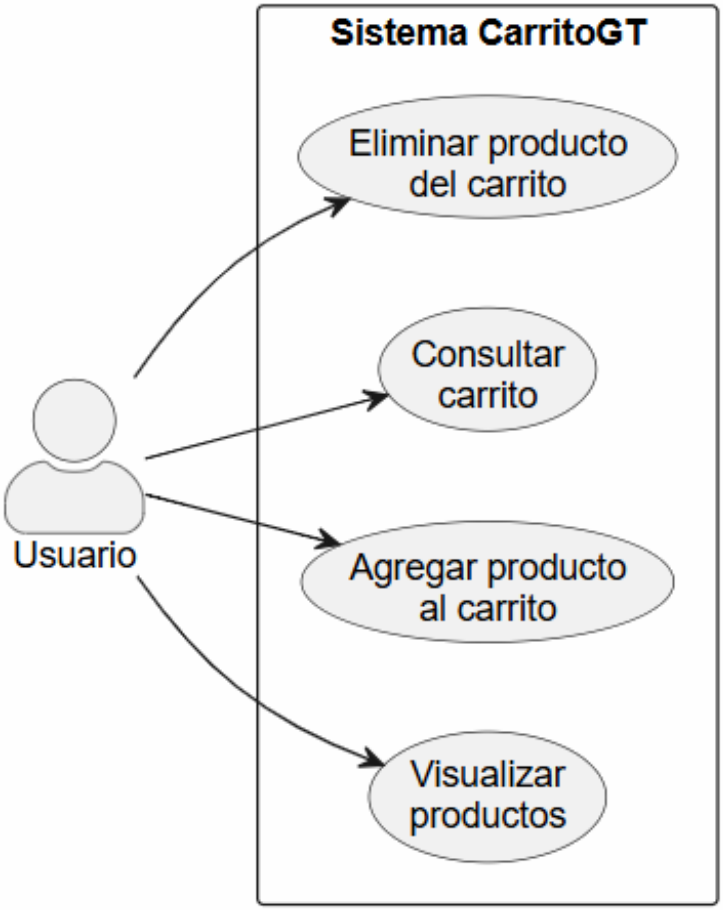


Figura 10. Diagrama de casos de uso del sistema CarritoGT.

7. Diagramas de arquitectura

7.1. Arquitectura de la versión PHP

La versión original del proyecto utiliza PHP para ejecutar la lógica de la aplicación y una base de datos MySQL o MariaDB para almacenar información. Apache recibe las solicitudes del navegador y permite acceder a los archivos de la aplicación.

Diagrama UML de Despliegue – Sistema CarritoGT

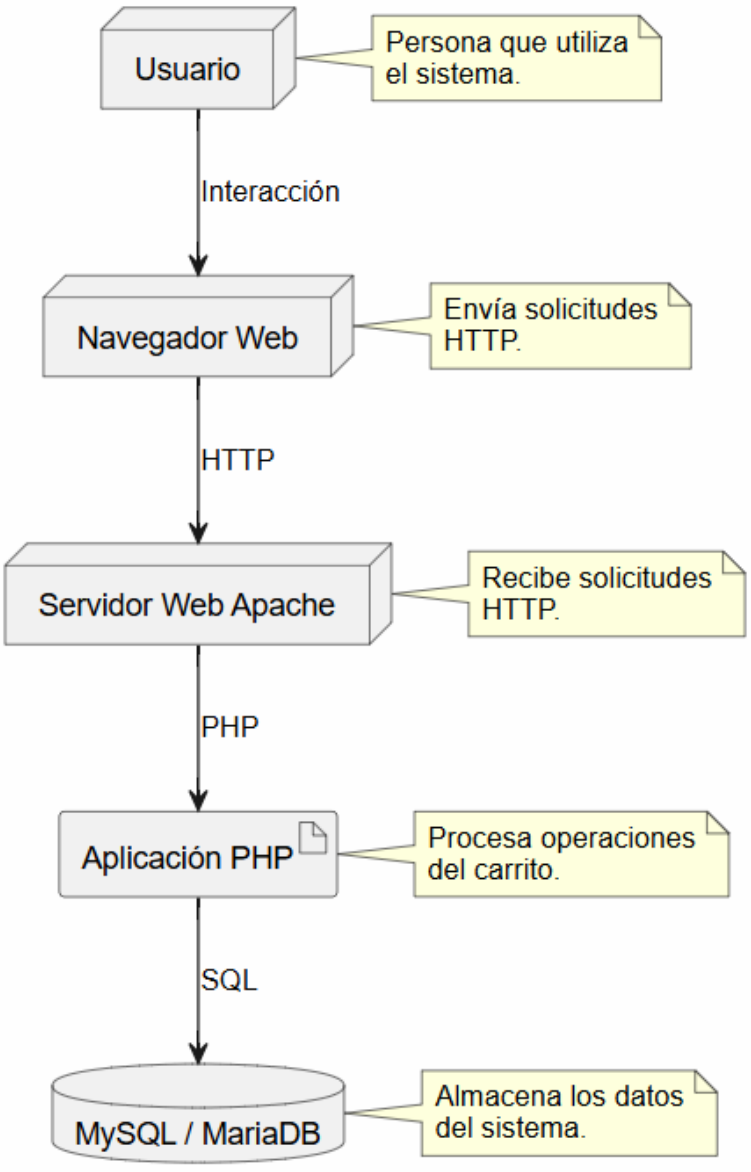


Figura 11. Arquitectura general de la versión PHP.

7.2. Arquitectura de la versión Spring Boot

La versión desarrollada en Java utiliza una arquitectura monolítica organizada por capas. Esto significa que los componentes forman parte de una misma aplicación, pero cada capa tiene una responsabilidad diferente.

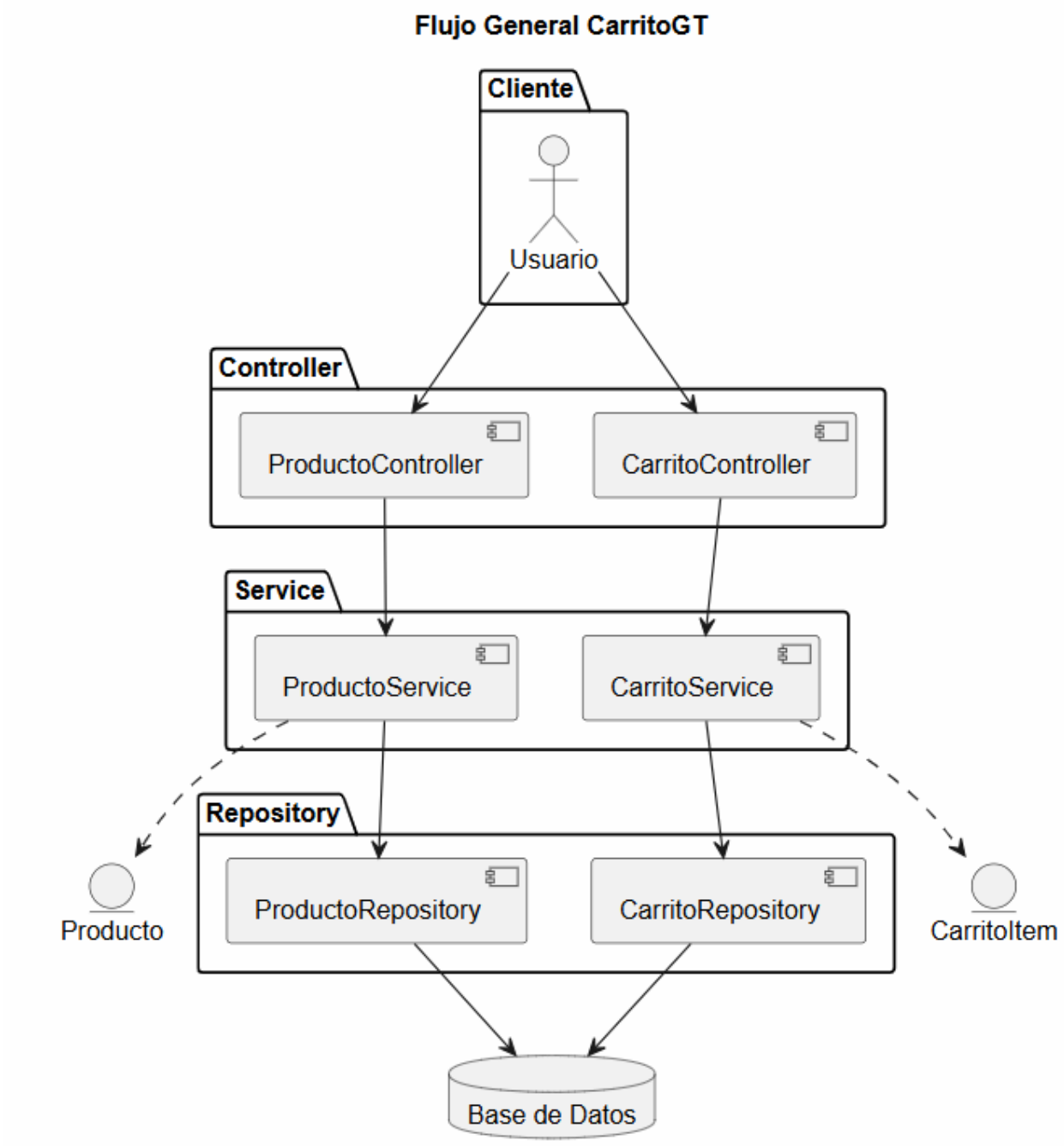


Figura 12. Arquitectura por capas de Spring Boot.

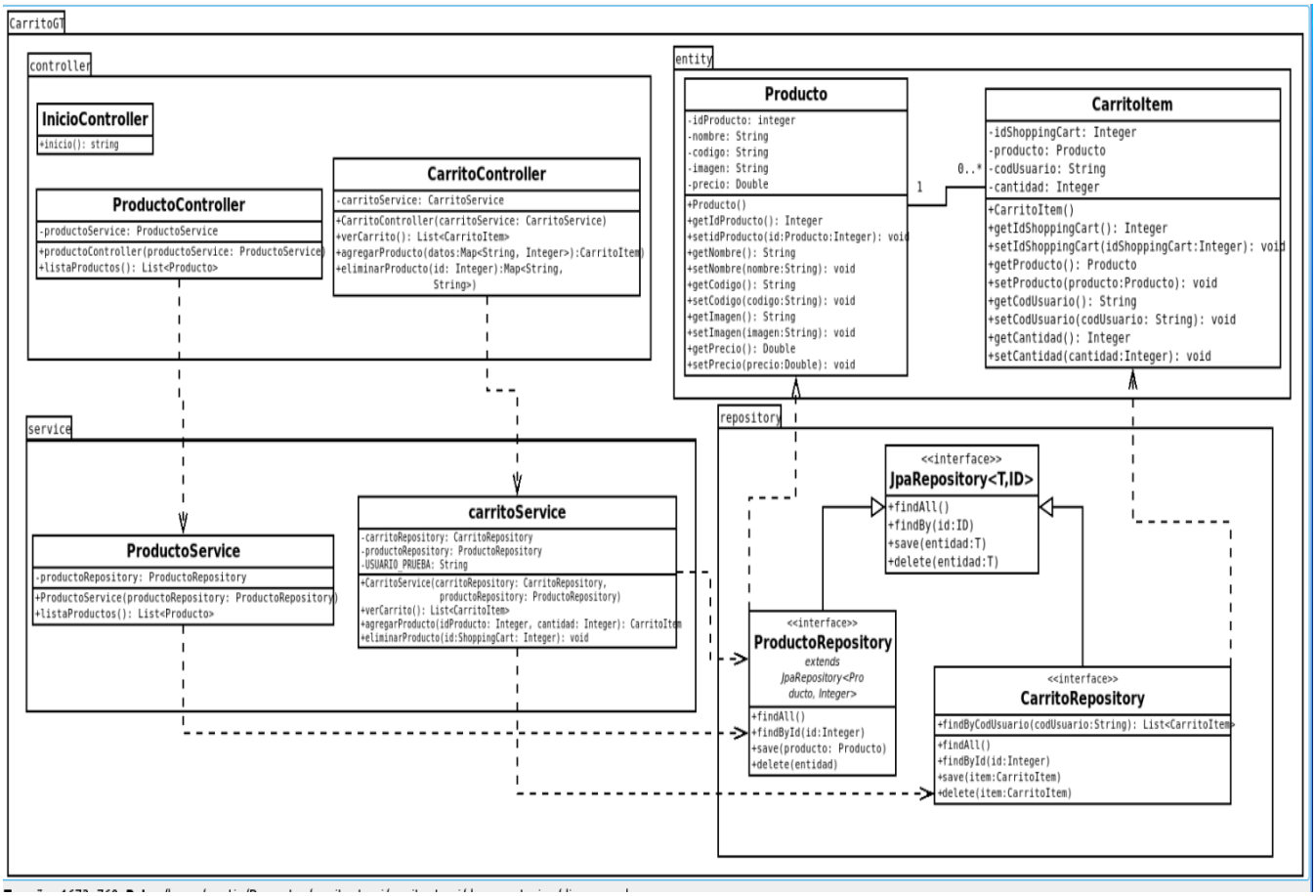


Figura 13. Diagrama UML de clases elaborado con Dia.

8. Diccionario de datos

8.1. Descripción

El diccionario de datos describe las tablas, los campos y las relaciones utilizadas para almacenar la información del sistema. Para CarritoGT, el diseño debe respetar el requisito de utilizar dos tablas y contar con entre cuatro y cinco productos de prueba, incluyendo sus imágenes.

Como en el proyecto se manejan las entidades Producto y CarritoItem, la siguiente propuesta puede servir para documentar la base de datos. Antes de entregar, debes comparar los nombres y tipos de campos con el script SQL real que te proporcionó el docente y con la configuración de tu aplicación.

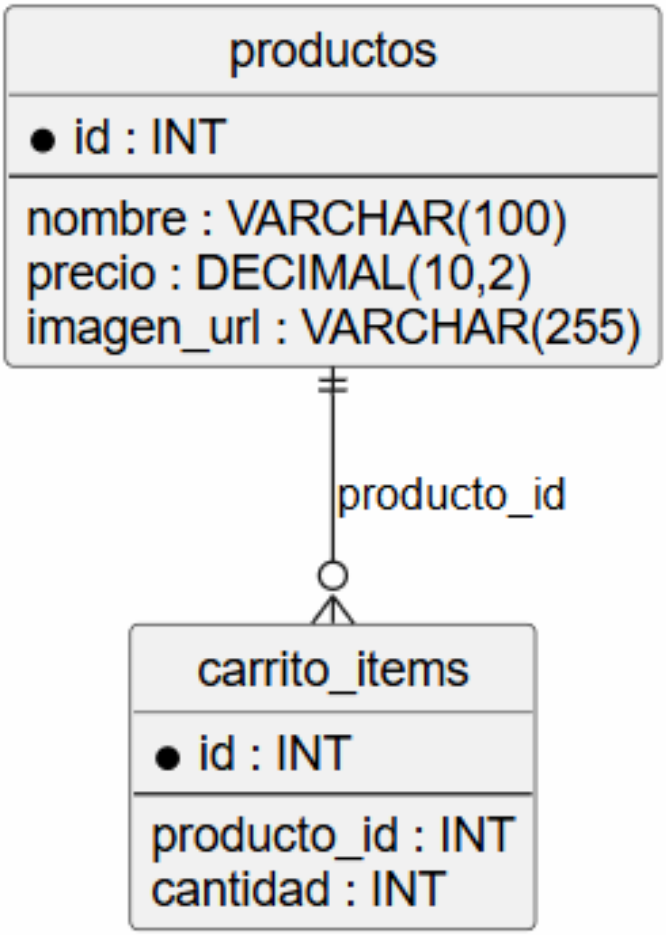


Figura 14. Script SQL de creación de las tablas.

8.4. Relación entre las tablas

La relación propuesta es de uno a muchos: un producto puede estar asociado a diferentes registros del carrito, mientras que cada registro del carrito hace referencia a un producto.

Esta relación se representa de la siguiente manera:

-productos.id → carrito_items.producto_id

La implementación exacta puede variar según el script SQL del docente. Si la tabla real no tiene una columna cantidad, o si utiliza otros nombres, debes documentar los campos reales en lugar de esta propuesta.

9. Contrato de la API REST

9.1. Descripción

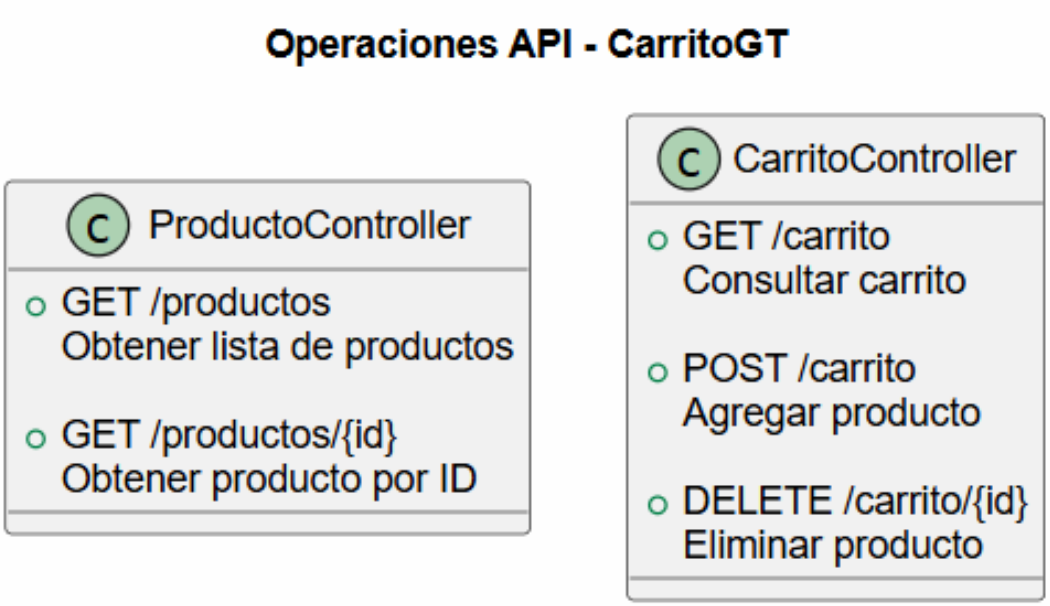
La API REST permite que un cliente se comunique con el servidor mediante solicitudes HTTP. En CarritoGT, Spring Boot procesa las solicitudes y devuelve respuestas que pueden representarse en formato JSON.

Los controladores del proyecto son ProductoController y CarritoController. Estos reciben las peticiones y se apoyan en los servicios para ejecutar las operaciones correspondientes.

El contrato de la API debe indicar las rutas disponibles, el método HTTP, la función de cada operación, los parámetros necesarios y los resultados esperados.

9.2. Endpoints que deben documentarse

La tabla siguiente es una plantilla funcional de referencia. Debes comprobar las rutas y los métodos exactos en ProductoController.java, CarritoController.java y en el archivo documentacion/API.md antes de presentarlos como endpoints implementados.



9.3. Ejemplo de respuesta JSON

Una respuesta ilustrativa para la consulta de productos podría tener esta estructura:

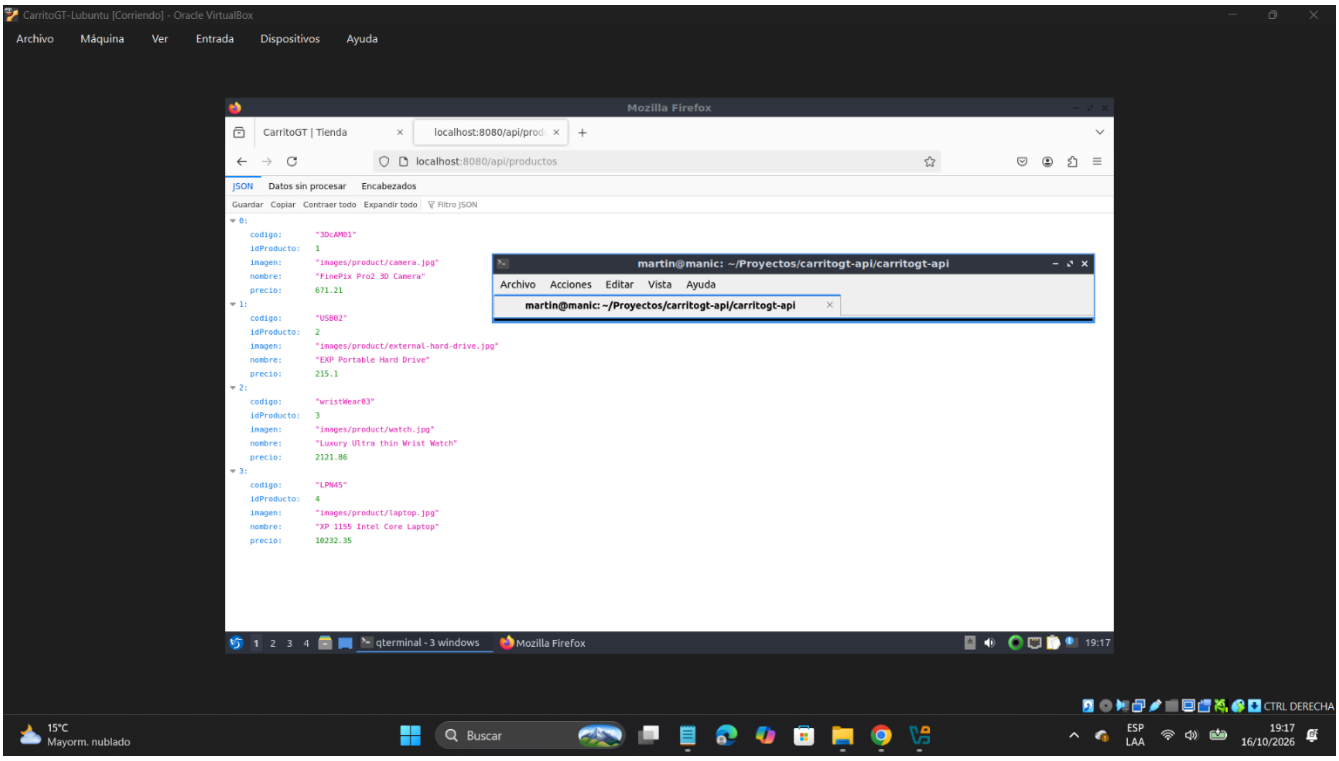


Figura 15. Conuslta de Productos con Respuesta JSON.

9.4. Parámetros y errores

Cada endpoint debe documentar los parámetros que recibe. Por ejemplo, una operación para agregar un producto podría requerir un identificador y una cantidad, dependiendo de cómo esté implementada.

La API también debe manejar errores como:

- Solicitar un producto que no existe.
- Enviar datos incompletos o incorrectos.
- Intentar eliminar un elemento inexistente.
- Encontrar un problema al consultar o modificar la información.

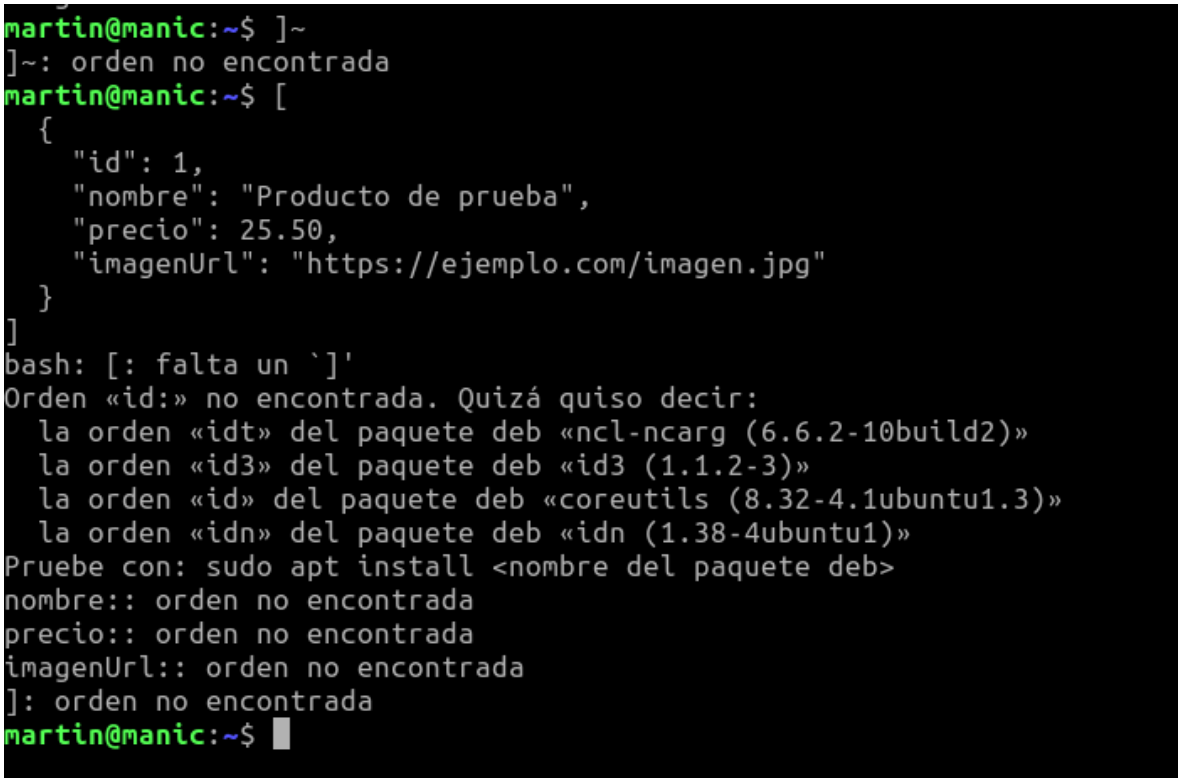


Figura 16. Prueba de Agregar un Producto al Carrito.

9.5. Pruebas de la API

Las solicitudes se pueden realizar desde el navegador, cURL o Postman, según el tipo de operación.

Ejemplo de una solicitud GET a la dirección base local:

curl -i http://localhost:8080/

Este comando sirve únicamente para comprobar la ruta raíz si está disponible. Para probar los productos o el carrito, reemplaza la dirección por el endpoint real de tu aplicación.

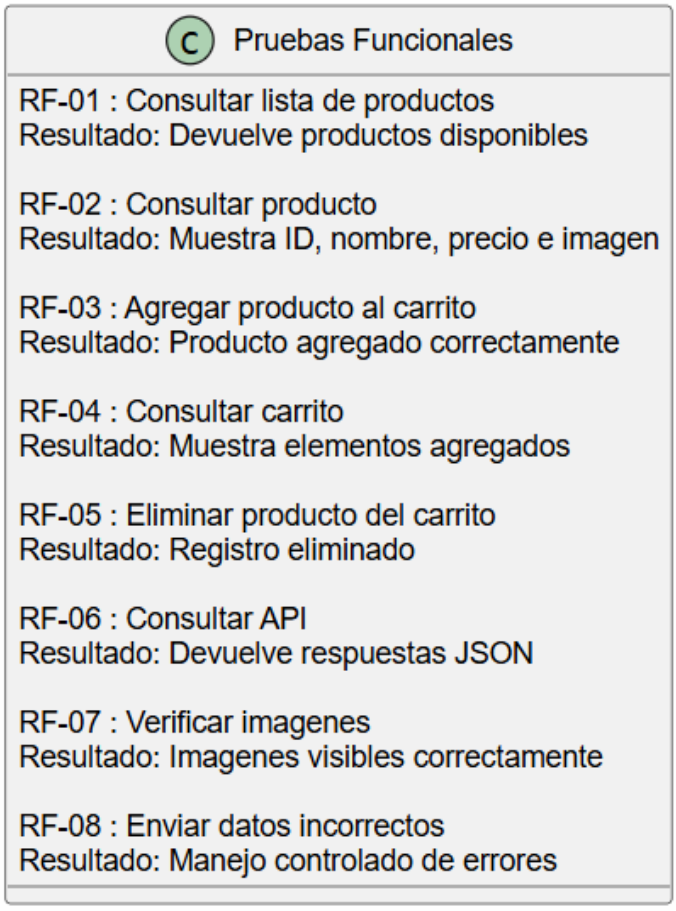
10. Evidencias de pruebas e integración

10.1. Objetivo

El objetivo de las pruebas es comprobar que las funciones principales de CarritoGT respondan correctamente y que la comunicación entre el cliente, el servidor y la base de datos se realice de acuerdo con los requisitos.

Se deben probar las operaciones individualmente y, posteriormente, verificar el flujo completo desde la consulta de productos hasta la eliminación de un artículo del carrito.

Casos de Prueba Funcionales - CarritoGT



Evidencia de pruebas automatizadas

Se ejecutaron las pruebas automatizadas del proyecto CarritoGT mediante Maven, utilizando el comando ./mvnw test. Esta ejecución permitió comprobar el resultado de las pruebas definidas en el proyecto e identificar posibles errores durante el proceso. El resultado de la ejecución se documenta mediante una captura de pantalla de la terminal.

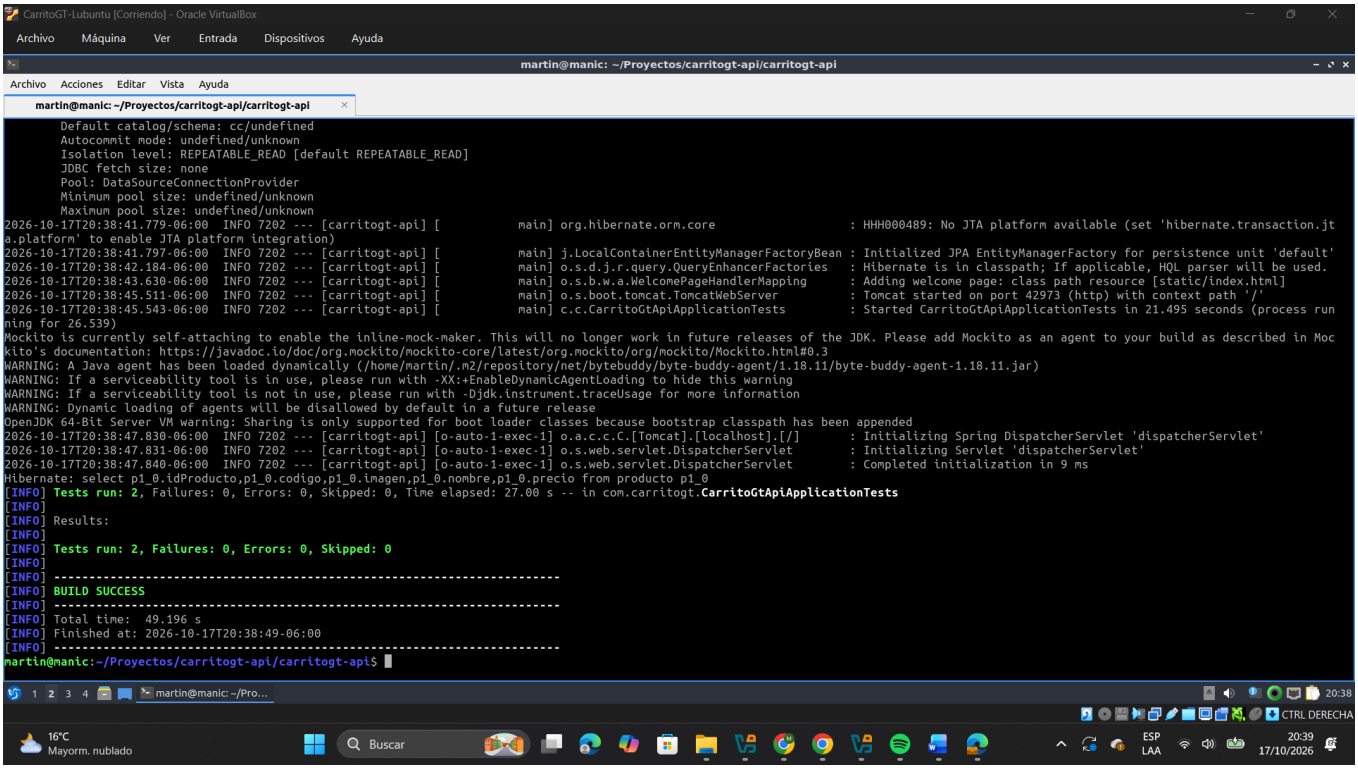


Figura 17.

10.4. Integración de los componentes

La integración se verifica al comprobar que el controlador reciba la solicitud, el servicio procese la operación y el repositorio acceda a los datos correspondientes. La respuesta debe regresar al cliente en el formato esperado.

Para la prueba de extremo a extremo, se recomienda seguir este orden:

1. Iniciar la aplicación Spring Boot.
2. Consultar los productos disponibles.
3. Agregar uno de los productos al carrito.
4. Consultar el carrito para verificar que se agregó.
5. Eliminar el producto.
6. Consultar nuevamente el carrito para verificar el resultado.
7. Revisar la respuesta del servidor y los mensajes de error, si los hubiera.

11. Manual de usuario

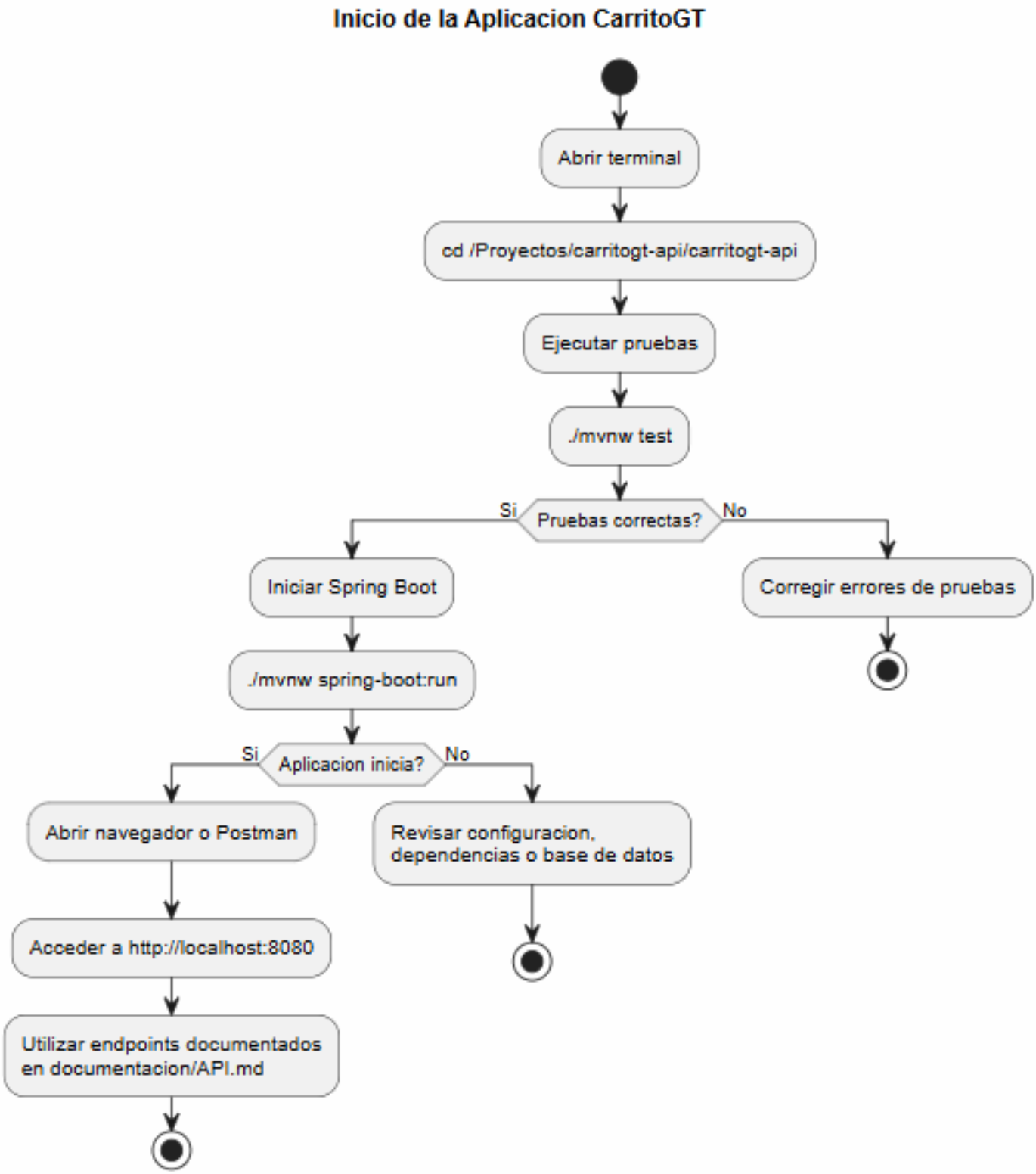
11.1. Introducción

CarritoGT es un proyecto académico orientado a la gestión básica de productos y de un carrito de compras. Su objetivo es permitir consultar productos, agregar artículos al carrito, revisar su contenido y eliminar elementos seleccionados.

La versión trabajada en Java utiliza Spring Boot para ofrecer servicios web mediante una API REST.

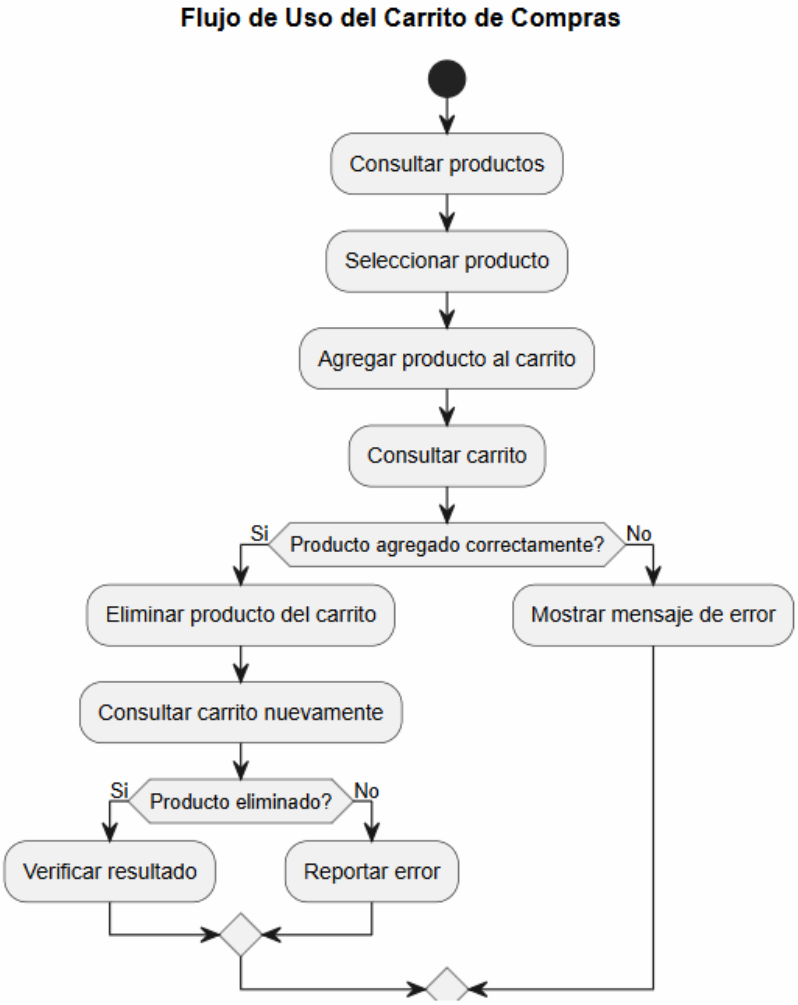
11.2. Requisitos para iniciar el sistema

Antes de ejecutar la aplicación, se debe comprobar que el entorno de desarrollo tenga Java y las herramientas necesarias para compilar el proyecto. También debe verificarse la configuración de la base de datos si la aplicación la utiliza durante su ejecución.


Inicio de la Aplicacion CarritoGT
Abrir terminal
cd /Proyectos/carritogt-api/carritogt-api
Ejecutar pruebas
./mvnw test
Pruebas correctas?
Si
No
Iniciar Spring Boot
Corregir errores de pruebas
./mvnw spring-boot:run
Aplicacion inicia?
Si
No
Abrir navegador o Postman
Revisar configuracion, dependencias o base de datos
Acceder a http://localhost:8080
Utilizar endpoints documentados en documentacion/API.md

11.4. Uso del carrito

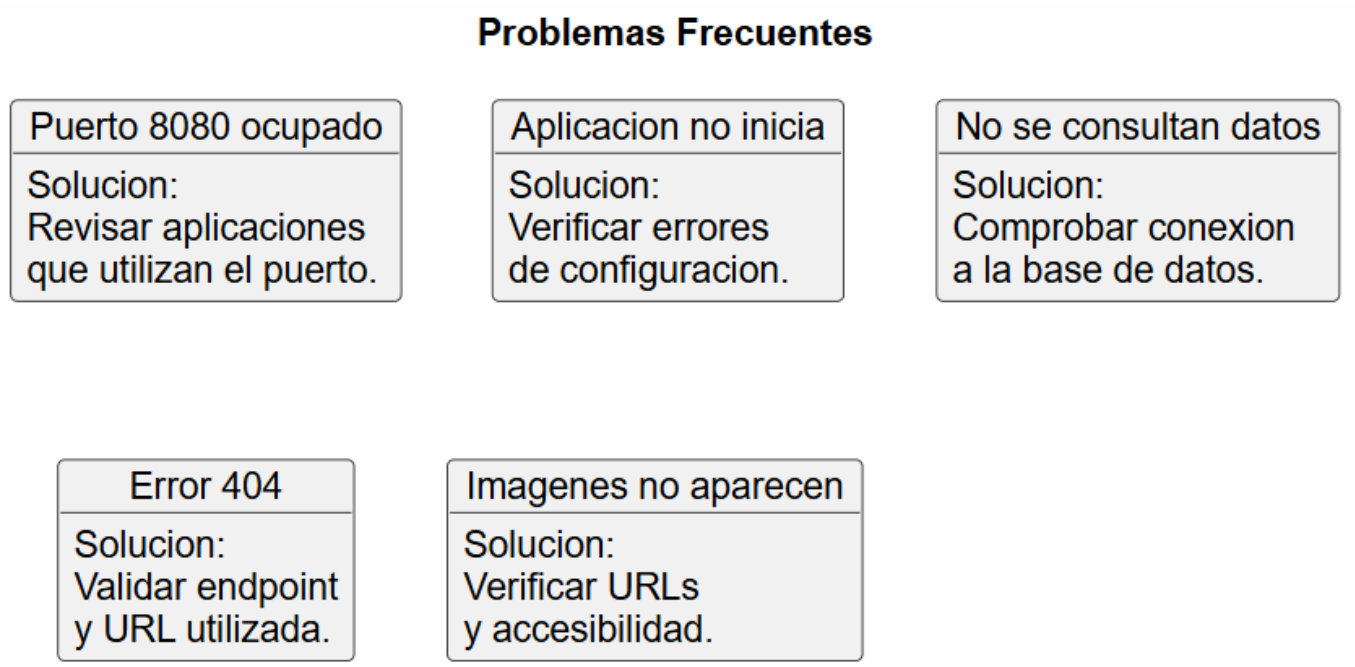
- Consultar productos: realizar una solicitud al endpoint de productos para conocer los artículos disponibles.
- Seleccionar un producto: identificar el artículo que se desea agregar.
- Agregar al carrito: enviar la solicitud correspondiente con los datos que requiera la API.
- Consultar el carrito: verificar que el producto se encuentre registrado.
- Eliminar un producto: enviar la solicitud de eliminación correspondiente.
- Verificar el resultado: consultar nuevamente el carrito para comprobar que el elemento se eliminó**.**

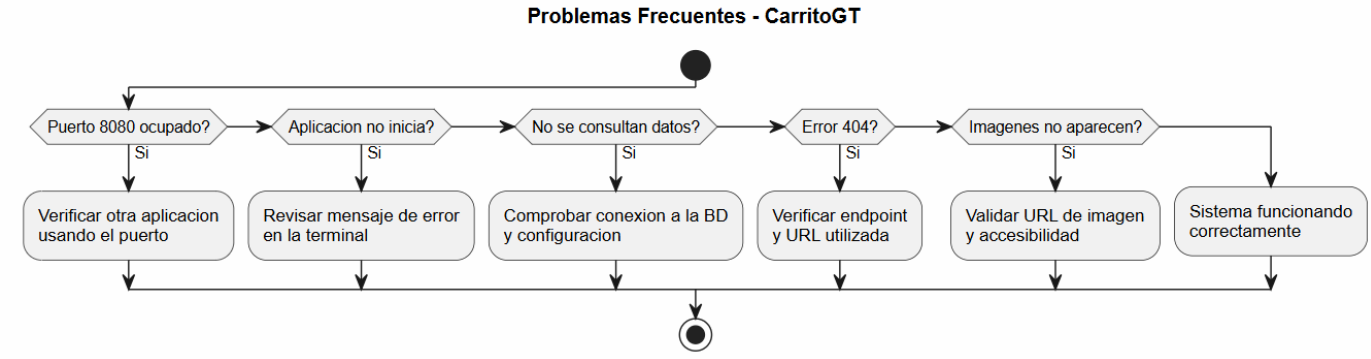


11.5. Cómo detener la aplicación

Si Spring Boot se está ejecutando en la terminal, se puede detener normalmente presionando Ctrl + C.

11.6. Problemas frecuentes





12. Bitácora de desarrollo

12.1. Descripción

La bitácora registra el avance del proyecto, los problemas encontrados, las decisiones tomadas y las soluciones aplicadas. También permite relacionar las actividades con los cambios realizados en Git y los archivos subidos al repositorio de GitHub.

En este caso, se documentan las actividades relacionadas con la organización del proyecto CarritoGT, la preparación del entorno, las pruebas y la documentación técnica.

12.2 Registro de actividades y problemas

Fecha	Actividad realizada	Evidencia o resultado
8/10/2026	Se revisaron las instrucciones del proyecto integrador y los requerimientos solicitados.	Requerimientos identificados y planificación inicial.
9/10/2026	Se planificó el entorno de virtualización para trabajar con Lubuntu y las herramientas necesarias.	Planificación del entorno de trabajo.
10/10/2026	Se preparó el entorno de desarrollo y se revisaron las herramientas necesarias para el proyecto.	Entorno de trabajo en preparación.
11/10/2026	Se comenzó el análisis del proyecto original desarrollado en PHP, identificando su estructura y funcionamiento.	Revisión de carpetas y archivos del proyecto original.
12/10/2026	Se organizó la estructura del proyecto CarritoGT para desarrollar la API utilizando Spring Boot y separación por capas.	Estructura inicial del proyecto Java.
13/10/2026	Se revisaron los requerimientos de los servicios REST y las operaciones necesarias para administrar los productos y el carrito.	Identificación de funcionalidades de la API.

14/10/2026	Se organizó la documentación técnica del proyecto, incluyendo la arquitectura y las pruebas.	Archivos de documentación preparados.
15/10/2026	Se continuó con el análisis de los requerimientos y la elaboración de los diagramas del sistema.	Avance de la documentación y del diseño.
16/10/2026	Se inicializó el proyecto CarritoGT con la API y la tienda web.	Commit **eb8723c**: Inicializa proyecto CarritoGT con API y tienda web.
16/10/2026	Se documentó la bitácora de trabajo y se registraron las actividades realizadas.	Commit **2b948d6**: Agrega bitácora de trabajo del proyecto.
16/10/2026	Se documentó la arquitectura por capas de CarritoGT.	Commit **ff4905a**: Documenta arquitectura por capas de CarritoGT.
16/10/2026	Se agregó una prueba automatizada para consultar los productos de la API.	Commit **c8e6720**: Agrega prueba automatizada para consultar productos.
16/10/2026	Se documentaron las pruebas automatizadas de la API.	Commit **4256959**: Documenta pruebas automatizadas de la API.
17/10/2026	Se agregaron evidencias relacionadas con las pruebas y el funcionamiento del proyecto.	Commit **b71fad2**: Agrega evidencias de pruebas y funcionamiento.
17/10/2026	Se agregó el diagrama UML de clases a la documentación del proyecto.	Commit **0e792c9**: Agrega diagrama UML de clases.
17/10/2026	Se continuó con la configuración del VirtualHost de Apache para ejecutar el proyecto PHP original.	Se identificó que debía utilizarse la carpeta **public** del proyecto como **DocumentRoot**.
17/10/2026	Se preparó la ejecución de las pruebas automatizadas mediante Maven.	Ejecución pendiente de confirmar con el resultado de la terminal.